



ClickHouse Support Engineer — Lab Assignment (Replicated Setup)

Objective:

Evaluate the candidate's ability to deploy, troubleshoot, and support a **replicated ClickHouse cluster** using either **ZooKeeper-based replication** or **ClickHouse Operator on Kubernetes**.



Environment Options (choose one):

Option A: Manual Setup (ZooKeeper-based)

- Deploy a **3-node ClickHouse cluster** with shared replication using **ZooKeeper** (can be local VMs, Docker containers, or cloud VMs)
- Include **1 ZooKeeper node** (or a 3-node ensemble if time permits)

Option B: Kubernetes + ClickHouse Operator

- Use Minikube, Kind, or any K8s environment
 - Deploy the **Altinity ClickHouse Operator**
 - Create a **replicated ClickHouseCluster** with at least 2 shards × 2 replicas
-

Part 1: Cluster Deployment (20 pts)

1. Deploy the ClickHouse cluster using Option A or B.
2. Confirm replication is working:
 - Create a replicated table on both replicas
 - Insert from one node, query from the other
3. Show ZooKeeper state or CRD status confirming health.



Deliverables:

- YAMLs or setup scripts
 - `system.replicas` and `system.parts` query output
 - Screenshot or logs proving replication
-

Part 2: Schema Design & Ingestion (20 pts)

1. Create a replicated table for logs:

```
CREATE TABLE logs ON CLUSTER '{cluster}'
```

```
(
```

```
timestamp DateTime,
```

```
service_name String,  
host String,  
log_level String,  
message String  
)  
ENGINE = ReplicatedMergeTree('/clickhouse/tables/{shard}/logs', '{replica}')  
PARTITION BY toYYYYMM(timestamp)  
ORDER BY (service_name, timestamp);
```

2. Load ~1M synthetic log entries into the table (Python/Go script, Kafka, or `clickhouse-client`).
3. Confirm records replicate properly across nodes.

✓ Deliverables:

- DDL statements
 - Ingestion script
 - Output from `SELECT count(*)` on each replica
-

Part 3: Query Performance & Troubleshooting (25 pts)

1. Write optimized queries for:
 - Error counts per service
 - Traffic per host
 - Logs over time (histogram/grouped per hour)
2. Enable `log_queries = 1` and monitor query metrics.
3. Simulate a slow query (e.g., heavy `ORDER BY`, missing partition filters), explain cause using:
 - `system.query_log`
 - `EXPLAIN`
 - `system.parts`, `system.merges`

✓ Deliverables:

- SQL queries
 - Analysis output
 - Fixes or suggested indexes/engine changes
-

Part 4: Failover and Recovery (25 pts)

1. Simulate replica failure:
 - In Docker: kill a node
 - In K8s: delete a pod
2. Insert data during outage, bring node back online.
3. Verify automatic syncing (check `system.replication_queue`, `system.replicas`, `system.mutations`).

✓ Deliverables:

- Failure simulation steps
 - Logs before and after
 - Output showing recovery and consistent data
-

Part 5: Bonus (Optional – 10 pts)

Choose one:

- Monitor the cluster using Prometheus + Grafana
 - Use Keeper instead of ZooKeeper
 - Set up replicated Kafka ingestion via [Kafka Engine + Materialized View](#)
 - Use [clickhouse-backup](#) to snapshot and restore one replicated table
-



Submission Checklist

- Cluster setup (YAMLs, scripts)
 - Table DDLs
 - SQL queries and outputs
 - Screenshots or logs for replication, failover, recovery
 - A short README explaining your choices and findings
-